

National University of Computer and Emerging Sciences



Lab Manual 06 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 6	4
Objectives:.....	4
Activities:	4
Activity 1 (Examples):	4
# Basic Game Tree Node Representation	4
# Minimax Algorithm (Recap)	4
Example Tree	4
# Python Code	4
# Minimax with Alpha–Beta Pruning.....	5
# Hill Climbing Algorithm - Finding maximum value in a landscape	6
Activity 2 (Searching Algorithms):.....	6
Alpha–Beta Pruning for Minimax Algorithm	6
Alpha–Beta Values	6
Alpha (α)	6
Beta (β)	6
Pruning Condition	7
Pruning Rules	7
At MAX Node	7
At MIN Node.....	7
Hill Climbing Algorithm	7
Important Concepts.....	7
State	7
Neighbour	7
Evaluation Function.....	7
Hill Climbing Algorithm Steps	8
Variants of Hill Climbing	8
Simple Hill Climbing	8
Steepest-Ascent Hill Climbing	8
Stochastic Hill Climbing	8
Limitations of Hill Climbing	8
Local Maximum.....	8
Plateau	8
Ridge	8
Techniques to Improve Hill Climbing.....	8
Activity 3 (Problem Statements / Tasks):.....	8

1. Minimax with Alpha–Beta Pruning	8
2. Alpha–Beta Pruning Efficiency Experiment.....	9
3. Hill Climbing for Maximum Value	9
4. Hill Climbing with Random Restart	10
5. Hill Climbing for Function Optimization	10

Lab Manual 6

Objectives:

- ✓ Game Playing using Minimax Algorithm with Alpha–Beta Pruning
- ✓ Hill Climbing Algorithm for optimization problems

Activities:

Activity 1 (Examples):

Basic Game Tree Node Representation

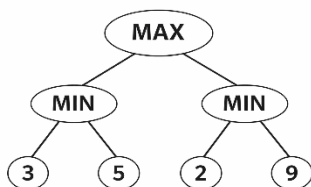
```
class GameNode:
    def __init__(self, value=None, children=None):
        self.value = value
        self.children = children if children else []
```

Minimax Algorithm (Recap)

```
def minimax(node, is_max):
    if not node.children:
        return node.value

    if is_max:
        best = float('-inf')
        for child in node.children:
            value = minimax(child, False)
            best = max(best, value)
        return best
    else:
        best = float('inf')
        for child in node.children:
            value = minimax(child, True)
            best = min(best, value)
        return best
```

Example Tree



Python Code

```
leaf1 = GameNode(3)
leaf2 = GameNode(5)
leaf3 = GameNode(2)
leaf4 = GameNode(9)
```

```

min1 = GameNode(children=[leaf1, leaf2])
min2 = GameNode(children=[leaf3, leaf4])

root = GameNode(children=[min1, min2])

print("Optimal Value:", minimax(root, True))

```

Minimax with Alpha-Beta Pruning

```

def minimax_alpha_beta(node, depth, alpha, beta, is_max):
    if not node.children:
        return node.value

    if is_max:
        best = float('-inf')

        for child in node.children:
            value = minimax_alpha_beta(child, depth+1, alpha, beta, False)
            best = max(best, value)

            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = float('inf')

        for child in node.children:
            value = minimax_alpha_beta(child, depth+1, alpha, beta, True)
            best = min(best, value)

            beta = min(beta, best)

            if beta <= alpha:
                break

        return best

print("Optimal Value with Alpha-Beta:",
      minimax_alpha_beta(root, 0, float('-inf'), float('inf'), True))

```

Hill Climbing Algorithm - Finding maximum value in a landscape

```
def hill_climbing(problem, start):
    current = start

    while True:
        neighbors = problem[current]

        # stop if no neighbors exist
        if not neighbors:
            return current

        best_neighbor = max(neighbors)

        if best_neighbor <= current:
            return current

        current = best_neighbor

landscape = {
    2: [3, 4],
    3: [5, 6],
    4: [6],
    5: [7],
    6: [7],
    7: []
}
print("Local Maximum:", hill_climbing(landscape, 2))
```

Activity 2 (Searching Algorithms):

Alpha-Beta Pruning for Minimax Algorithm

The basic Minimax algorithm explores all nodes of the game tree. For games with large branching factors, this becomes computationally expensive.

Alpha-Beta Pruning improves Minimax by:

- Avoiding exploration of branches that cannot affect the final decision
- Reducing the number of nodes evaluated
- Producing the same result as Minimax but faster

Alpha-Beta Values

Two parameters are maintained during search.

Alpha (α)

Best value MAX can guarantee so far.

Initial value: $\alpha = -\infty$

Beta (β)

Best value MIN can guarantee so far.

Initial value: $\beta = +\infty$

Pruning Condition

If at any time:

$$\alpha \geq \beta$$

Remaining nodes in that branch do not need to be explored.

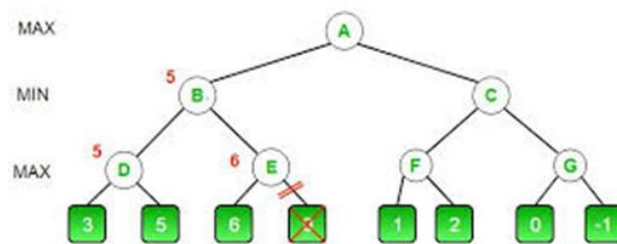
Pruning Rules

At MAX Node

If $current_value \geq \beta$, stop exploring further children. This is called Beta Cut-off.

At MIN Node

If $current_value \leq \alpha$, stop exploring further children. This is called Alpha Cut-off.



Hill Climbing Algorithm

Local search algorithms are a class of search techniques that focus on finding an optimal solution by iteratively improving a single candidate solution. Unlike traditional search algorithms that explore a tree or graph and maintain a record of all explored paths, local search algorithms maintain only the current state in memory. These algorithms are particularly useful for optimization problems where the goal is to find the best possible state rather than the exact path to that state.

Hill Climbing is one of the simplest form of local search algorithm that attempts to improve the current solution by moving to a better neighbouring state. It is widely used for problems with large state spaces.

Hill Climbing evaluates only the neighbouring states of the current position and moves to the one that appears to be the best. If no neighbour improves the current state, the algorithm stops, potentially at a local maximum rather than the global optimum.

It is analogous to: *A person climbing a hill in fog, always moving upward.*

Important Concepts

State

A candidate solution.

Example: Position on a hill.

Neighbour

A state reachable from the current state.

Example: Nearby positions on the hill.

Evaluation Function

Measures how good a state is.

$$f(state)$$

Higher value $\rightarrow$ Better State

Hill Climbing Algorithm Steps

1. Choose an initial state
2. Repeat:
 - Evaluate current state
 - Generate neighbours
 - Choose best neighbour
3. If best neighbour is better:
 - Move to neighbour
4. Otherwise:
 - Stop (local optimum)

Variants of Hill Climbing

Simple Hill Climbing

Examines neighbours one by one.

Steepest-Ascent Hill Climbing

Evaluates all neighbours and chooses the best.

Stochastic Hill Climbing

Chooses random improving neighbour.

Limitations of Hill Climbing

Hill climbing may get stuck in:

Local Maximum

Better than neighbours but not global best.

Plateau

Flat area with equal values.

Ridge

Narrow path difficult for greedy moves.

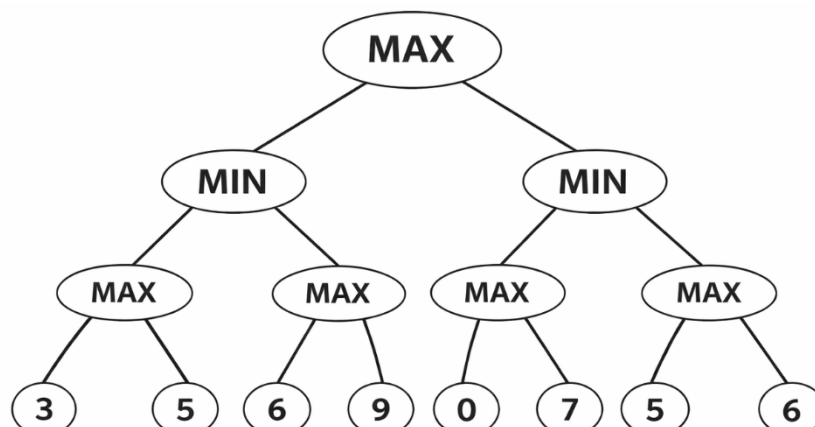
Techniques to Improve Hill Climbing

- Random Restart Hill Climbing
- Sideways Moves
- Stochastic Hill Climbing

Activity 3 (Problem Statements / Tasks):

1. *Minimax with Alpha-Beta Pruning*

Construct the following game tree using the GameNode class.



Requirements

- Construct the above game tree using the GameNode class.
- Implement the Minimax algorithm.
- Implement Minimax with Alpha–Beta Pruning.
- Display:
 - Optimal value at the root node
 - Value returned by each MIN node
 - Value returned by each MAX node
- Display which branches were pruned during Alpha–Beta pruning.

Compare

Count and display:

- Number of leaf nodes evaluated using Basic Minimax
- Number of leaf nodes evaluated using Alpha–Beta Pruning

Explain:

- Why Alpha–Beta Pruning explores fewer nodes.

2. *Alpha–Beta Pruning Efficiency Experiment*

Using the same game tree from Problem 1, analyse the efficiency of Alpha–Beta pruning.

Requirements

- Implement Basic Minimax.
- Implement Minimax with Alpha–Beta Pruning.
- Modify the program to count the number of nodes visited.

Display:

- Nodes visited using Basic Minimax
- Nodes visited using Alpha–Beta Pruning
- Nodes saved due to pruning

Questions

Students must explain:

- Which algorithm evaluated fewer nodes?
- Why does Alpha–Beta Pruning improve performance?
- Does Alpha–Beta change the final optimal value?

3. *Hill Climbing for Maximum Value*

Consider the following state landscape:

```
landscape = {  
    1: [2, 3],  
    2: [4, 5],  
    3: [5],  
    4: [7],  
    5: [6],  
    6: [8],  
    7: [],  
    8: []  
}
```

Requirements

- Implement the Hill Climbing algorithm.
- Start the search from state 1.
- At each step:
 - Evaluate neighbours
 - Move to the best neighbour.

Display

- Visited states during the search
- Current state at each step
- Final local maximum reached

Explain:

- Why the algorithm stops at the final state.

4. *Hill Climbing with Random Restart*

Hill Climbing can sometimes get stuck in local maxima.

Modify the Hill Climbing algorithm to implement Random Restart Hill Climbing.

Requirements

- Run Hill Climbing multiple times with different starting states.
- Randomly select starting states from the landscape.
- Run the algorithm 5 times.

Display

- Starting state for each run
- Local maximum obtained in each run
- Best solution among all runs
- Total number of restarts

Explain

- Why Random Restart can produce better solutions than a single run.

5. *Hill Climbing for Function Optimization*

Consider the mathematical function:

$$f(x) = -(x - 3)^2 + 9$$

This function has a maximum value at $x = 3$.

Requirements

- Implement Hill Climbing to find the maximum value.
- Generate neighbours using:
 - $x + 1$
 - $x - 1$
- Start the search from:
 - $x = 0$

Display

- Current value of x at each step
- Value of $f(x)$
- Neighbour evaluated
- Final maximum value obtained

Expected Result

The algorithm should converge near:

$$x = 3$$

$$f(x) = 9$$